

Test 02 Practice - print me, write me out (avoid/fight looking up answers when writing)

1. (10 PTS.) These questions deal with transforming a VList collection; consider the following book record:

```
record Book(String title, int pages, int year) { }

// ----- in a void main() {
var books = VList.of(          // inferred type for 'books': VList<Book>
    new Book("The Martian", 220, 2019),
    new Book("Project Hail Mary", 410, 2023),
    new Book("Biological War: A Scenario", 432, 2026),
    new Book("Dust", 416, 2013)
);
```

(a) (3 PTS.) use `filter` to produce a list containing only books with at least 300 pages (use the books list above)

(b) (3 PTS.) use `map` to produce a new `VList<Book>` where each book has the same title and year, but its number of pages is increased by 10

(c) (4 PTS.) use both `filter` and `map` to produce the titles of books published in 2020 or later

2. (5 PTS.) These questions also deal with `VList`; continue to use the books list from the previous question.

(a) (3 PTS.) state both the contents and the inferred type for `xs`, `ys`, and `zs`

```
var xs = books.filter(b -> b.pages() > 250);
```

```
var ys = books.map(b -> b.pages());
```

```
var zs = books.map(b -> b.title());
```

(b) (2 PTS.) suppose the following is evaluated:

```
var recent = books.filter(b -> b.year() >= 2020);
```

Does this operation modify books? Briefly explain.

3. (7 PTS.) This question deals with functional interfaces.

```
Function<Integer, String> f = n -> "value=" + n;  
Predicate<String> p      = s -> s.length() < 6;
```

(a) (2 PTS.) state both the parameter type and return type for f and p

(b) (3 PTS.) show one valid call to f; show two calls to p: one producing true and one producing false; be sure to call the correct method (either apply or test)

(c) (2 PTS.) what is the main difference between a Function<A,B> and a Predicate<A>? A few words.

4. (6 PTS.) This question deals with Lambda (λ) expressions.

For each description below: i) write a λ -expression and ii) write the type of the λ -expression (it'll either be a Predicate<..> or a Function<..>).

(a) (2 PTS.) take an Integer and return whether it is negative

(b) (2 PTS.) take a String and return its length plus 1

(c) (2 PTS.) take an Integer and return the String "even" when the integer received is even and "odd" otherwise

5. (5 PTS.) This is more of an object modeling question; suppose a shipping application supports two kinds of `Parcel`s:

- a `Box`, with a weight and volume
- an `Envelope`, with a weight and whether it is certified

Every `Parcel` should provide a `double weight()` method. Write a sealed interface `Parcel` and suitable record implementations for `Box` and `Envelope`.

6. (7 PTS.) This question deals with the `Maybe` type; consider the following:

```
Maybe<Integer> firstNegative(VList<Integer> values)
```

(a) (3 PTS.) implement `firstNegative`; loop over the values and return the first negative value wrapped in `Some`; return `None` if the list contains no negative value

(b) (2 PTS.) suppose `result` has type `Maybe<Integer>`; complete the following switch expression:

```
var message = switch (result) {  
    // TODO  
};
```

For `Some(-3)`, message should be "first negative: -3". For `None`, message should be "no negative value".

(c) (2 PTS.) state what each call should return and explain briefly why `Maybe<Integer>` is useful here.

```
firstNegative(VList.of(4, 8, -3, -7))
```

```
firstNegative(VList.of(4, 8, 3, 7))
```

7. (10 PTS.) This question deals with a command-line MVC application for a playlist. The model contains entities modeling a song and a playlist (i.e.: a Playlist object holds a list of Song objects):

```
// assume these public records are in separate files
// in a model package
public record Song(String title, int seconds) { }

public record Playlist(VList<Song> songs) {
    public VList<Song> longerThan(int seconds) {
        return songs.filter(s -> s.seconds() > seconds);
    }
}
```

The command-line view is:

```
class CliView {
    public void showSongs(VList<Song> songs) {
        // TODO
    }
}
```

(a) (4 PTS.) implement showSongs such that it prints the title of each song, one per line (use a foreach style loop)

(b) (3 PTS.) the controller has fields named model and view. Complete showLongSongs so that it asks the model for songs longer than the given number of seconds and sends the result to the view.

```
class Controller {
    private Playlist model;
    private CliView view;

    // constructor omitted

    public void showLongSongs(int seconds) {
        // TODO
    }
}
```

(c) (3 PTS.) Timmy instead writes the following version of showLongSongs in the Controller:

```
public void showLongSongs(int seconds) {
    var songs = model.longerThan(seconds);

    for (var song : songs) {
        IO.println(song.title());
    }
}
```

This works, but it is not a particularly good MVC design.

What responsibility is the controller taking on that should belong to the view? Briefly describe how you

would change the method.